
Computer Science and Engineering Department, SVNIT, Surat
M.Tech. I – Semester I
Design and Analysis of Algorithm (CSDS103)
Lab Assignment 4
Divide and Conquer

1 Introduction to Divide and Conquer

The **Divide and Conquer** paradigm is a fundamental algorithmic design technique based on multi-branched recursion. It breaks a complex problem into smaller, independent subproblems of the same type, solves them recursively, and subsequently combines their individual solutions to construct the solution to the original problem. The paradigm adheres to three canonical phases:

1. **Divide:** Partition the input problem into two or more disjoint subproblems of smaller size.
2. **Conquer:** Solve the subproblems recursively. If the subproblem size is sufficiently small (the base case), solve it directly without further subdivision.
3. **Combine:** Merge the solutions of the subproblems into a unified global solution.

In this laboratory assignment, we analyze and implement two representative applications of the Divide and Conquer strategy:

- **Problem 1 (Counting Inversions and Sorting Comparison):** Simultaneous sorting and inversion counting via an augmented Merge Sort in $O(n \log n)$ time, compared against an $O(n^2)$ Brute Force method, Quick Sort, and Insertion Sort.
 - **Problem 2 (Fast Exponentiation):** Logarithmic computation of powers a^n in $O(\log n)$ time via Divide and Conquer, contrasted against linear repeated multiplication and naive recursive branching.
-

2 Problem 1: Counting Inversions and Sorting Comparison

2.1 Problem Statement and Definitions

Given an array $A = [a_1, a_2, \dots, a_n]$ of n distinct elements, an **inversion** is defined as a pair of indices (i, j) such that:

$$i < j \quad \text{and} \quad A[i] > A[j] \quad (1)$$

The inversion count measures how far an array is from being monotonically sorted. An array sorted in ascending order contains exactly 0 inversions, whereas an array sorted in strictly descending order contains the maximum possible number of inversions:

$$\binom{n}{2} = \frac{n(n-1)}{2} = \Theta(n^2) \quad (2)$$

Counting inversions is central to various computational domains, including collaborative filtering algorithms, rank correlation coefficients (e.g., Kendall tau distance), and genomic distance metrics.

2.2 Algorithmic Strategies

1. **Divide and Conquer Inversion Counter ($O(n \log n)$):** Modifies Merge Sort by dividing the array into two halves:

- Count inversions entirely contained within the left subarray (inv_{left}).
- Count inversions entirely contained within the right subarray (inv_{right}).
- Count split inversions where i belongs to the left subarray and j belongs to the right subarray (inv_{split}).

Because both halves are recursively sorted, when merging the two halves, if $A_{\text{left}}[i] > A_{\text{right}}[j]$, then every remaining element in A_{left} from index i to the end of the left subarray is strictly greater than $A_{\text{right}}[j]$. Hence, $\text{len}(A_{\text{left}}) - i$ split inversions are counted in $O(1)$ amortized steps per comparison.

2. **Brute Force Inversion Counter ($O(n^2)$):** Examines every distinct pair of indices (i, j) with $i < j$ via nested loops and increments a counter whenever $A[i] > A[j]$.

3. **Baseline Sorting Benchmarks:**

- *Quick Sort*: Standard partition-exchange sort with average-case time $\Theta(n \log n)$.
- *Insertion Sort*: Incremental in-place sorting algorithm with average-case time $\Theta(n^2)$.

2.3 Python Implementations

The complete Python implementations from DAA_Lab4.ipynb are listed below:

Listing 1: Problem 1 Algorithm Implementations

```

1 # 1. Divide and Conquer: Merge Sort and Count Inversions
2 def count_inversions_dc(arr):
3     if len(arr) <= 1:
4         return arr, 0
5     mid = len(arr) // 2
6     left, inv_left = count_inversions_dc(arr[:mid])
7     right, inv_right = count_inversions_dc(arr[mid:])
8
9     merged = []
10    i = j = inv_split = 0
11    while i < len(left) and j < len(right):
12        if left[i] <= right[j]:
13            merged.append(left[i])
14            i += 1
15        else:
16            merged.append(right[j])
17            inv_split += len(left) - i
18            j += 1

```

```

19     merged.extend(left[i:])
20     merged.extend(right[j:])
21     return merged, inv_left + inv_right + inv_split
22
23 # 2. Brute Force Inversion Counting
24 def count_inversions_bf(arr):
25     inv = 0
26     n = len(arr)
27     for i in range(n):
28         for j in range(i + 1, n):
29             if arr[i] > arr[j]:
30                 inv += 1
31     return inv
32
33 # 3. Baseline Sorting Algorithms for Comparison
34 def quick_sort(arr):
35     if len(arr) <= 1:
36         return arr
37     pivot = arr[len(arr) // 2]
38     left = [x for x in arr if x < pivot]
39     mid = [x for x in arr if x == pivot]
40     right = [x for x in arr if x > pivot]
41     return quick_sort(left) + mid + quick_sort(right)
42
43 def insertion_sort(arr):
44     a = arr.copy()
45     for i in range(1, len(a)):
46         key = a[i]
47         j = i - 1
48         while j >= 0 and a[j] > key:
49             a[j + 1] = a[j]
50             j -= 1
51         a[j + 1] = key
52     return a

```

2.4 Verification on Sample Input

The correctness of the implementations was verified using the reverse-sorted sample array $[8, 4, 2, 1]$. Because the elements are in strictly descending order, the total number of inversions is $\binom{4}{2} = 6$.

Listing 2: Console Output for Sample Verification

```

Original Array: [8, 4, 2, 1]
Divide & Conquer: Sorted = [1, 2, 4, 8], Inversions = 6
Brute Force:      Inversions = 6

```

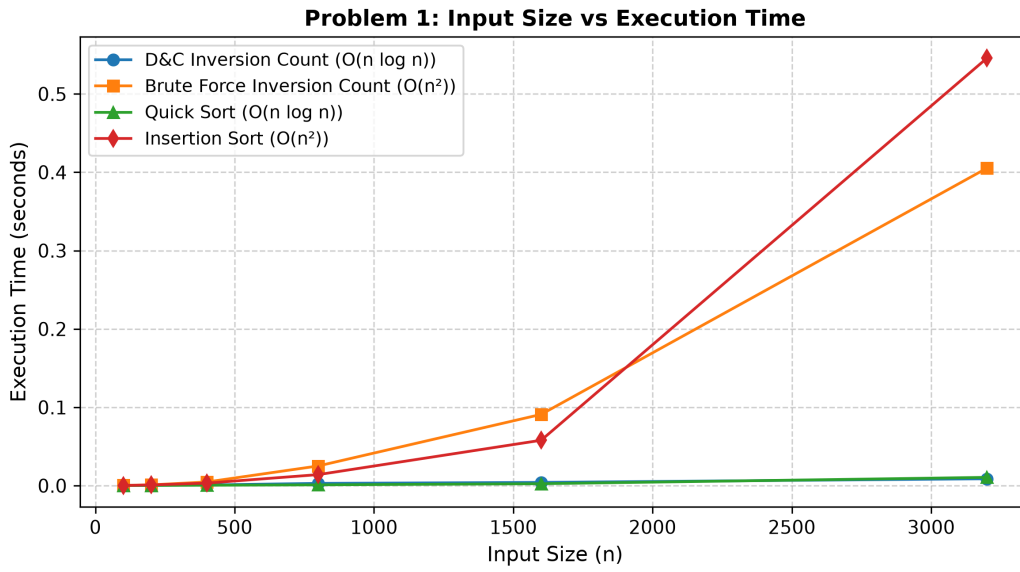
Both algorithms correctly identify 6 inversions, confirming implementation fidelity.

2.5 Experimental Performance Benchmarking

To empirically compare the scaling behavior, randomly generated integer arrays of sizes $n \in \{100, 200, 400, 800, 1600, 3200\}$ were tested. The execution times (measured in seconds using high-resolution monotonic clocks) are summarized in Table 1.

Table 1: Problem 1: Empirical Execution Times Across Input Sizes (seconds)

Input Size (n)	D&C Inversions (s)	Brute Force (s)	Quick Sort (s)	Insertion Sort (s)
100	0.000289	0.000328	0.000144	0.000194
200	0.000424	0.001013	0.000271	0.000912
400	0.000876	0.004677	0.000545	0.003223
800	0.002905	0.025146	0.001013	0.014090
1600	0.004140	0.090939	0.002461	0.057929
3200	0.008791	0.405262	0.010584	0.545639

Figure 1: Problem 1: Input Size (n) vs Execution Time (seconds) comparing Divide & Conquer, Brute Force, Quick Sort, and Insertion Sort.

2.6 Theoretical Complexity and Experimental Analysis

1. Divide and Conquer Complexity:

- Recurrence Relation:**

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n) \quad (3)$$

where $2T(n/2)$ accounts for the two recursive subproblem invocations and $\Theta(n)$ represents the linear merging step where split inversions are tabulated.

- Master Theorem Analysis:** For $T(n) = aT(n/b) + f(n)$ with $a = 2$, $b = 2$, and $f(n) = \Theta(n^d)$ with $d = 1$:

$$\log_b a = \log_2 2 = 1 = d \quad (4)$$

By Case 2 of the Master Theorem:

$$T(n) = \Theta(n^{\log_b a} \log n) = \Theta(n \log n) \quad (5)$$

- **Auxiliary Space:** Requires $\Theta(n)$ auxiliary space to store temporary subarrays during the merge phase, along with $O(\log n)$ recursive call stack frames.

2. Brute Force Complexity:

- **Time Complexity:** The total number of pairwise comparisons is:

$$\sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \sum_{i=1}^{n-1} (n-i) = \frac{n(n-1)}{2} = \Theta(n^2) \quad (6)$$

Each comparison involves an $O(1)$ inequality check. Thus, time complexity is $\Theta(n^2)$.

- **Auxiliary Space:** Uses strictly $\Theta(1)$ auxiliary space as it performs pairwise scans in-place.

3. Comparison with Baseline Sorting Algorithms:

- *Quick Sort:* Average time complexity $\Theta(n \log n)$, auxiliary space $O(\log n)$ recursion stack.
- *Insertion Sort:* Average/Worst-case time complexity $\Theta(n^2)$, auxiliary space $O(1)$ in-place.

4. **Experimental vs. Theoretical Correspondence:** As illustrated in Figure 1, the quadratic curves for Brute Force Inversion Counting and Insertion Sort grow steeply (405.2 ms and 545.6 ms at $n = 3200$). In stark contrast, Divide and Conquer Inversion Counting and Quick Sort maintain nearly linear trajectories (8.8 ms and 10.6 ms at $n = 3200$), proving that the $O(n \log n)$ divide-and-conquer strategy offers an order-of-magnitude practical speedup for large datasets.

3 Problem 2: Fast Exponentiation

3.1 Problem Statement and Definitions

The exponentiation problem requires evaluating a^n for a base $a \in \mathbb{R}$ (or $\mathbb{Z}$) and a non-negative integer exponent $n \in \mathbb{Z}_{\geq 0}$. For large values of n , standard linear evaluation becomes prohibitively inefficient.

3.2 Algorithmic Approaches

1. **Repeated Multiplication ($O(n)$):** Directly computes $a^n = \underbrace{a \times a \times \cdots \times a}_{n \text{ times}}$ using an iterative loop executing n sequential multiplications.
2. **Naive Recursive Exponentiation ($O(n)$):** Implements the piecewise mathematical identity without caching subproblems:

$$a^n = \begin{cases} 1 & \text{if } n = 0 \\ a^{n/2} \times a^{n/2} & \text{if } n \text{ is even} \\ a \times a^{n-1} & \text{if } n \text{ is odd} \end{cases} \quad (7)$$

Because both branches of $a^{n/2}$ are evaluated independently without memoization, redundant recursive subproblems cause exponential branching in the recurrence tree.

3. **Divide and Conquer Exponentiation ($O(\log n)$):** Evaluates the subproblem half = $a^{\lfloor n/2 \rfloor}$ exactly once and stores it in a temporary variable:

$$a^n = \begin{cases} 1 & \text{if } n = 0 \\ (\text{half})^2 & \text{if } n \text{ is even} \\ a \times (\text{half})^2 & \text{if } n \text{ is odd} \end{cases} \quad (8)$$

4. **Iterative Divide and Conquer (Binary Exponentiation):** Evaluates powers iteratively by expressing the exponent n in binary form: $n = \sum_{k=0}^{\lfloor \log_2 n \rfloor} b_k 2^k$, eliminating recursive call stack overhead completely.

3.3 Python Implementations

Listing 3: Problem 2 Exponentiation Implementations

```

1 # 1. Repeated Multiplication
2 def power_iterative(a, n):
3     res = 1
4     for _ in range(n):
5         res *= a
6     return res
7
8 # 2. Recursive Exponentiation (Naive Branching from PDF Relation)
9 def power_recursive_naive(a, n):
10     if n == 0:

```

```

11     return 1
12     if n % 2 == 0:
13         return power_recursive_naive(a, n // 2) * power_recursive_naive(a, n // 2)
14     else:
15         return a * power_recursive_naive(a, n - 1)
16
17 # 3. Divide and Conquer Exponentiation (Subproblem Reused)
18 def power_dc(a, n):
19     if n == 0:
20         return 1
21     half = power_dc(a, n // 2)
22     if n % 2 == 0:
23         return half * half
24     else:
25         return a * half * half
26
27 # 4. Iterative Divide and Conquer (Binary Exponentiation)
28 def power_dc_iterative(a, n):
29     res = 1
30     base = a
31     while n > 0:
32         if n % 2 == 1:
33             res *= base
34             base *= base
35             n //= 2
36     return res

```

3.4 Verification on Sample Input

The functions were verified for $a = 2, n = 10$, where the exact analytical result is $2^{10} = 1024$.

Listing 4: Console Output for Exponentiation Verification

```

Computing 2^10:
Repeated Multiplication : 1024
Naive Recursive         : 1024
Divide and Conquer      : 1024
Iterative D&C           : 1024

```

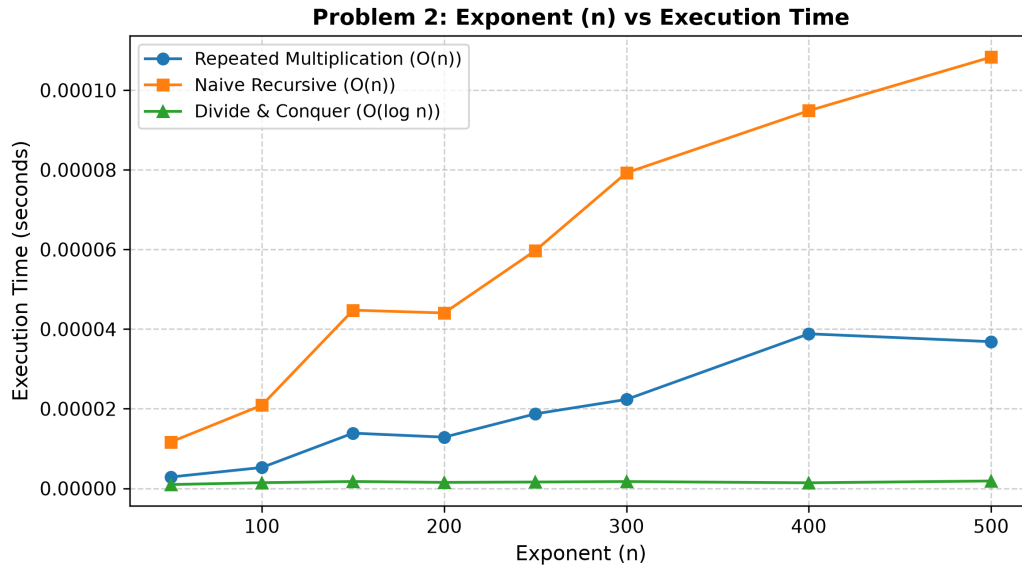
All four implementations produce the identical and correct output of 1024.

3.5 Experimental Performance Benchmarking

To assess empirical runtimes, the three principal algorithms were benchmarked for base $a = 2$ and exponents $n \in \{50, 100, 150, 200, 250, 300, 400, 500\}$. Each test was averaged over 100 iterations. Table 2 presents the recorded runtimes.

Table 2: Problem 2: Average Execution Times Across Exponents ($a = 2$)

Exponent (n)	Repeated Mult (s)	Naive Recursive (s)	Divide & Conquer (s)
50	2.86×10^{-6}	1.16×10^{-5}	0.97×10^{-6}
100	5.25×10^{-6}	2.09×10^{-5}	1.43×10^{-6}
150	1.39×10^{-5}	4.48×10^{-5}	1.73×10^{-6}
200	1.29×10^{-5}	4.41×10^{-5}	1.51×10^{-6}
250	1.87×10^{-5}	5.97×10^{-5}	1.60×10^{-6}
300	2.23×10^{-5}	7.93×10^{-5}	1.71×10^{-6}
400	3.88×10^{-5}	9.49×10^{-5}	1.40×10^{-6}
500	3.68×10^{-5}	1.08×10^{-4}	1.86×10^{-6}

Figure 2: Problem 2: Exponent (n) vs Execution Time (seconds) comparing Repeated Multiplication, Naive Recursive, and Divide & Conquer.

3.6 Theoretical Analysis and Lab Questions

3.6.1 Question 1: Recurrence Relation for Divide and Conquer

The Divide and Conquer algorithm computes $a^{\lfloor n/2 \rfloor}$ once and performs at most two multiplications (squaring half $\times$ half, and multiplying by a if n is odd):

$$T(n) = T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + \Theta(1) \quad (9)$$

Using the Master Theorem $T(n) = aT(n/b) + f(n)$:

- $a = 1$ (one subproblem solved recursively)
- $b = 2$ (subproblem size halved)

$$\bullet f(n) = \Theta(1) = \Theta(n^0) \implies d = 0$$

Evaluating the exponent ratio:

$$\log_b a = \log_2 1 = 0 = d \quad (10)$$

Since $\log_b a = d$, by Case 2 of the Master Theorem:

$$T(n) = \Theta(n^d \log n) = \Theta(\log n) \quad (11)$$

The algorithm executes in strictly logarithmic time with respect to the exponent n .

3.6.2 Question 2: Time and Auxiliary Space Complexity Summary

Table 3 summarizes the computational complexities of all three approaches:

Table 3: Complexity Summary for Exponentiation Methods

Approach	Time Complexity	Auxiliary Space Complexity
Repeated Multiplication	$\Theta(n)$	$\Theta(1)$
Naive Recursive Exponentiation	$\Theta(n)$	$\Theta(\log n)$ stack frames
Divide and Conquer Exponentiation	$\Theta(\log n)$	$\Theta(\log n)$ stack frames
Iterative Divide and Conquer	$\Theta(\log n)$	$\Theta(1)$

3.6.3 Question 3: Effect of Repeatedly Computing Subproblems in the Naive Approach

In the naive recursive definition:

$$\text{power_recursive_naive}(a, n//2) * \text{power_recursive_naive}(a, n//2)$$

both invocations evaluate the exact same expression independently.

- For an even exponent n , the recurrence is $T(n) = 2T(n/2) + O(1)$. By the Master Theorem ($a = 2, b = 2, d = 0 \implies \log_2 2 = 1 > 0$), $T(n) = \Theta(n)$.
- The total number of recursive function calls grows linearly with n (approximately $2n$ total invocations) rather than logarithmically.
- The Divide and Conquer algorithm caches the subproblem result in variable `half`, resolving the recurrence to $T(n) = T(n/2) + O(1)$, reducing the call tree depth to exactly $\lfloor \log_2 n \rfloor$ steps.

3.6.4 Question 4: Why Reducing Multiplications Drastically Affects Execution Time for Large n

When n is very large, the number of bits in the result a^n becomes enormous. Specifically, the bit length b of a^n is:

$$b \approx \log_2(a^n) = n \log_2 a \quad (12)$$

In real-world computer systems (such as Python's arbitrary-precision integers), multiplying two large b -bit numbers is **not** an $O(1)$ hardware instruction. Large integer multiplication takes super-linear bit complexity:

- Karatsuba algorithm: $O(b^{\log_2 3}) \approx O(b^{1.585})$
- FFT-based multiplication: $O(b \log b \log \log b)$

In repeated multiplication, performing n multiplications on growing integers requires accumulating bit-level overhead at every step, yielding an overall bit complexity of $O(n^2 \log^2 a)$. In contrast, Divide and Conquer performs only $\approx \log_2 n$ multiplications. The overwhelming majority of these multiplications occur on smaller intermediate bit lengths, achieving dramatic algorithmic acceleration.

3.6.5 Question 5: Iterative Divide and Conquer and Auxiliary Space Elimination

The recursive Divide and Conquer implementation requires $\lfloor \log_2 n \rfloor + 1$ stack frames, leading to $\Theta(\log n)$ auxiliary space. An iterative formulation (binary exponentiation) is implemented as:

Listing 5: Iterative Binary Exponentiation

```
1 def power_dc_iterative(a, n):
2     res = 1
3     base = a
4     while n > 0:
5         if n % 2 == 1:
6             res *= base
7             base *= base
8             n //= 2
9     return res
```

Auxiliary Space Impact:

- The iterative approach uses only two local scalar variables (**res** and **base**) and avoids recursion entirely.
- Auxiliary memory consumption is reduced from $\Theta(\log n)$ stack frames to strictly $\Theta(1)$ **auxiliary space**.

3.6.6 Question 6: Paradigm Comparison: Problem 1 vs Problem 2

Both problems exploit the Divide and Conquer framework, but they exhibit distinct structural characteristics across the three phases:

Table 4: Paradigm Comparison: Problem 1 vs Problem 2

Phase	Problem 1: Inversion Counting	Problem 2: Fast Exponentiation
Divide	Partitions array into 2 halves of size $n/2$.	Reduces exponent n into a single subproblem of size $\lfloor n/2 \rfloor$.
Conquer	Recursively solves both subproblems ($2T(n/2)$).	Recursively solves only one subproblem ($1T(n/2)$).
Combine	Merges the two sorted halves in linear time $\Theta(n)$ while counting split inversions.	Combines in $\Theta(1)$ time by squaring half (and multiplying by a if n is odd).
Overall Time	$\Theta(n \log n)$ via Master Theorem Case 2 ($a = 2, b = 2, d = 1$).	$\Theta(\log n)$ via Master Theorem Case 2 ($a = 1, b = 2, d = 0$).

3.6.7 Question 7: Theoretical Complexity vs. Experimentally Observed Wall-Clock Time

There is an inherent discrepancy between asymptotic theoretical complexity and experimental execution time:

1. **Theoretical Complexity (O, Θ, Ω):** Focuses on asymptotic rates of operation growth as input size approaches infinity ($n \rightarrow \infty$). It abstracts away architecture-dependent constants, CPU pipelining, and system overhead.
2. **Experimental Wall-Clock Time:** Measures physical elapsed time on actual hardware. It is influenced by:
 - *Constant factors:* Function call overhead in interpreted languages like Python (byte-code interpretation, dynamic typing).
 - *Memory hierarchy:* CPU L1/L2/L3 cache misses and RAM bandwidth during array allocation in merge sort.
 - *Operating system scheduling:* Context switching, timer tick resolution, and CPU frequency throttling (Turbo Boost / thermal throttling).

Despite these physical hardware variances, the empirical curves in Figures 1 and 2 closely corroborate the asymptotic orders of growth ($O(n^2)$ vs $O(n \log n)$ in Problem 1; $O(n)$ vs $O(\log n)$ in Problem 2).

4 Conclusion

In this laboratory assignment, the Divide and Conquer paradigm was implemented, validated, and evaluated on two fundamental problems. For inversion counting, augmentations to Merge Sort reduced the runtime from $\Theta(n^2)$ to $\Theta(n \log n)$. For exponentiation, evaluating subproblems once reduced multiplications from $\Theta(n)$ to $\Theta(\log n)$, with binary exponentiation achieving optimal $\Theta(1)$ auxiliary space. The theoretical derivations via the Master Theorem were corroborated by empirical timing benchmarks.